

[← Back to programme](#)[Module Overview](#)

7% Completed

[Assignments](#) ▾[← Optimal Transport](#)

Auction

[Reset](#) [Clone ▾](#)[INSTRUCTIONS](#) [FEEDBACK](#) [WEB-IDE](#)

Auction

Problem Statement

You are at an auction of NFT art bidding against robot players.

The auction is conducted in rounds. Each round consists in a sequence of N bids against M robot opponents, where N and M are fixed within a round but will change across the rounds of an auction.

An auction round is conducted as follows: The victory point of the current item is revealed, and then every participant will place a bid for the item. After that we determine the winner of the current item and then move on to the next item. This process is repeated for each item in the round. Each item will give the winner some victory points. The objective is to maximise victory points gained from the items in each auction round.

At the start of each auction round, each of the participants is given a fixed budget X , where X equals to 1. Any remaining budget not used in each round cannot be carried for future rounds and they will not provide any victory points. The budget will be reset at the start of each auction round.

We will follow an auction rule where all bidders will pay the bidding amount submitted for an item, regardless of whether they win the auction for that item. When there is a tie in the bidding amount, you will win the auction against robots.

You are required to develop a bidding strategy given the bidding information and auction results of previous items and keep track of the budget you have used in each round. The bidding strategy will return a bidding value which represents the amount you are willing to pay for the current item.

Computational Constraints

The number of robots M is an integer between 5 and 10

The number of items N is an integer between $2 * M$ and $6 * M$

Victory points of each item $V[k]$ is a positive float between 0 to 1 where k is the index of the item between 0 to $N-1$

Budget X is a constant set to 1

Input and Outputs

Input:

game_information, A dictionary containing the configuration of the auction, the current round number, total number of items to be auctioned in the round and the number of robots in the auction. The game information is not changed during the round. Example: `{'Round_Number': 1, 'Total_Number_Items': 10, 'Number_Robots': 5}`. This indicates that the first auction round has 10 items and bidding against 5 robots.

current_info, A dictionary containing the information of the current item to be auctioned. Example: `{'Remaining_Number_Items': 10, 'vp': 0.4, 'remaining_budget': 0.8, }`. This indicates that there are 10 items left (including the current item) in the auction round and the current item to be auctioned has 0.4 victory points. Your remaining budget is 0.8 for the current and rest of the items in the auction round.

last_auction_result, A dictionary containing the winner, price, and victory points for the last item being auctioned and the total and minimum bidding values submitted. Example: `{'winner': 'player', 'winning_bid': 0.5, 'vp': 1, 'total': 2.1, 'min': 0.2}`. This indicates that the player wins the bid of the last item with a bidding value of 0.5 and a victory point of 1. All the participants (you and the robots) submitted a bidding total of 2.1 for the last item. The minimal bidding value submitted is 0.2. For the first item auctioned in each round, the value of each key in this dictionary is set to None as follows `{'winner': None, 'winning_bid': None, 'vp': None, 'total': None, 'min': None}`.

The names of the winner for each item is represented by one of the following.

- 'player' if the player wins the item.
- 'Robot_X' if the Xth robot wins the item where X depends on the number of robots in that auction round.

player_information, A dictionary of information that you want to persist over the tournament

Output:

bidding_value, A float which represents the amount the player is willing to bid. It needs to be greater than or equal to zero and within the total budget the player has left in the round.

Scoring

For each auction, the score is the ratio of your total victory points with the total victory points of the best player. The score is a number between 0 to 1. The example below shows the details on how to calculate the score.

Test structure and time limits

- There are 50 auction rounds.
- The score of each test is the average of the scores for each of the auction round.
- Time limit is 30 seconds.

Examples

Example 1

Consider the following example of an auction round which consists of 20 items against 5 robots, we will show a sample input and output for an auction item and how to calculate the score at the end of the round.

$M = 5$

$N = 20$

Sample Input

game_information:

```
{'Round_Number': 1, 'Total_Number_Items': 20, 'Number_Robots': 5}
```

current_info:

```
{'Remaining_Number_Items': 17, 'vp': 0.6, 'remaining_budget': 0.8 }
```

last_auction_result:

```
{'winner': 'Robot_1', 'winning_bid': 0.5, 'vp': 0.7, 'total': 1.2, 'min': 0.1}
```

player_information:

```
{}
```

Sample Output

0.2

Explanation

There are 5 robots bidding against the player this round for a total of 20 items.

The Victory Point of the current item is 0.6.

The auction for the last item (3rd item) with victory point 0.7 is won by Robot_1 with a bid of 0.5.

The total bid submitted by the robots and player is 1.2 and the minimal bid submitted is 0.1.

The player has a remaining budget of 0.8 for the items left in this round. The player is going to submit a bid of 0.2 for the current item.

The above bidding process is repeated for each item in the round.

At the end of the auction round we calculate the victory points for each player based on the items they won.

Agent	Victory Points
player	5.6
Robot_1	5.2
Robot_2	4.5
Robot_3	6.3
Robot_4	7.0
Robot_5	3.5

Robot_4 has the highest total victory points of 7.0. The normalised score of the player is $5.6/7.0 = 0.8$.

Example Solution:

The following strategy bids equally for all items

```
def auction_bids(game_info, current_info, last_auction_result, player_information):
    bid_value = 0.999 / game_info['Total_Number_Items'] ## Rounding Error
    return bid_value
```